Quiz: Operators and Decorators

Total Score: $/2^2$

Printed Name:

Quiz rules:

1. You MAY use any printed or handwritten notes.
2. You MAY NOT use a computer or any other electronic device.

Problem 1. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def foo(xs):
    return foo(xs)
try:
    foo([])
except RecursionError:
    print('RecursionError')
except RuntimeError:
    print('RuntimeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 2. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
a = Vec(1, 2)
b = Vec(1, 2)
try:
    print('a == b:', a == b)
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 3. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
from dataclasses import dataclass
@dataclass
class Vec:
    x: int
    y: int
    def __rmul__(self, other):
        if not isinstance(other, int):
            return NotImplemented
        return Vec(other * self.x, other * self.y)
try:
    z = 3 * Vec(1, 2)
    print('z=', z)
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 4. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
# HINT:
# this problem is *almost* exactly the same as one of
# the problems in your review packet;
# the only difference is that a single character has
# been changed somewhere

def add_arithmetic(cls):
    def __add__(self, other):
        return cls(self.x + other.x, self.y * other.y)
    def __radd__(self, other):
        return self
    cls.__add__ = __add__
    cls.__radd__ = __radd__
    return cls

@add_arithmetic
class Vec:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __repr__(self):
        return f'Vec({self.x}, {self.y})'

xs = [Vec(1, 2), Vec(3, 4), Vec(5, 6)]
print('sum(xs) =', sum(xs))
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $12 \;/\; 16 = 0.75$

LLM Model Performance

